

ARM Cross Compilation + QEMU Setup — Purpose of Every Command

This setup is used in real embedded Linux and firmware development environments such as:

- Hewlett Packard Enterprise iLO firmware
- Cisco routers/switches
- NVIDIA Jetson boards
- Raspberry Pi Foundation ARM devices
- Automotive ECU firmware
- IoT gateways
- Embedded Linux systems

STEP 1 — Install ARM Toolchain + QEMU

```
sudo apt update && sudo apt install -y \  
build-essential \  
gcc-arm-linux-gnueabi \  
binutils-arm-linux-gnueabi \  
qemu-user \  
qemu-system-arm \  
git \  
wget \  
curl \  
libncurses-dev \  
flex \  
bison \  
openssl \  
libssl-dev \  
device-tree-compiler \  
cpio \  
unzip \  
rsync \  
python3 \  
python3-pip \  
bc \  
libelf-dev
```

PURPOSE OF EACH COMMAND / PACKAGE

Package	Purpose	Why Needed
<code>sudo apt update</code>	Updates Ubuntu package database	Downloads latest package list before installation
<code>apt install -y</code>	Installs packages automatically	<code>-y</code> avoids confirmation prompts
<code>build-essential</code>	Installs GCC, Make, libc headers	Required for compiling C/C++ programs
<code>gcc-arm-linux-gnueabi</code> <code>binutils-arm-linux-gnueabi</code>	ARM cross compiler	Compiles code on x86 Ubuntu for ARM devices
<code>binutils-arm-linux-gnueabi</code>	ARM linker/assembler tools	Provides <code>ld</code> , <code>objdump</code> , <code>nm</code> , <code>ar</code>
<code>qemu-user</code>	ARM binary emulator	Runs ARM applications directly on Ubuntu
<code>qemu-system-arm</code>	Full ARM machine emulator	Emulates complete ARM hardware systems
<code>git</code>	Version control tool	Used to clone Linux/U-Boot projects
<code>wget</code>	File downloader	Downloads firmware/source packages
<code>curl</code>	API/network tool	Used for HTTP requests and automation
<code>libncurses-dev</code>	Terminal UI library	Needed for Linux kernel <code>menuconfig</code>
<code>flex</code>	Lexical analyzer generator	Used during Linux kernel builds

bison	Parser generator	Required for compiler/kernel parsing
openssl	SSL utility	Used for certificates and signing
libssl-dev	OpenSSL development headers	Needed when compiling secure applications
device-tree-compiler	Compiles Device Tree files	ARM kernels use DTB hardware descriptions
cpio	Archive utility	Used in initramfs/rootfs creation
unzip	ZIP extractor	Extracts downloaded packages
rsync	Fast file synchronization	Used in build/deployment workflows
python3	Python runtime	Required by many build systems
python3-pip	Python package installer	Installs Python dependencies
bc	Command-line calculator	Linux kernel build dependency
libelf-dev	ELF binary library	Needed for kernel/module builds
u-boot-tools	U-Boot utilities	Creates boot images for embedded boards

WHAT THIS STEP ACHIEVES

After installation:

Component	Purpose
ARM Cross Compiler	Build ARM binaries
QEMU Emulator	Run ARM binaries without hardware
Embedded Linux Tools	Kernel + firmware development
U-Boot Utilities	Bootloader image creation
Python + Build Tools	Automation/build scripting

STEP 2 — Verify Installation

`arm-linux-gnueabi-hf-gcc --version`

Purpose

Checks whether ARM cross compiler installed correctly.

What Happens

- Displays GCC ARM compiler version
- Confirms toolchain path is configured

Expected Output

`arm-linux-gnueabi-hf-gcc (Ubuntu 12.x.x)`

`qemu-arm --version`

Purpose

Checks whether ARM emulator installed correctly.

What Happens

- Displays QEMU version
- Verifies ARM runtime emulator availability

Expected Output

`qemu-arm version 8.x.x`

STEP 3 — Environment Validation

Component	Status	Meaning
Cross compiler	✓	ARM code compilation works
ARM runtime	✓	ARM binaries can execute
QEMU emulator	✓	ARM environment emulation works
Ubuntu setup	✓	System ready for embedded training

STEP 4 — Create ARM Test Program

Create Training Directory

```
mkdir -p ~/ilo-training/test
```

Purpose

Creates project directory structure.

What Happens

- `mkdir` → create directory
- `-p` → create parent folders automatically

Result

```
~/ilo-training/test
```

```
cd ~/ilo-training/test
```

Purpose

Moves terminal into project directory.

Create Source File

nano hello.c

Paste:

```
#include <stdio.h>
```

```
int main() {  
    printf("ARM test success\n");  
    return 0;  
}
```

PURPOSE OF THIS C PROGRAM

Code	Purpose
<code>#include <stdio.h></code>	Includes standard input/output library
<code>main()</code>	Program entry point
<code>printf()</code>	Prints message to terminal
<code>return 0</code>	Indicates successful execution

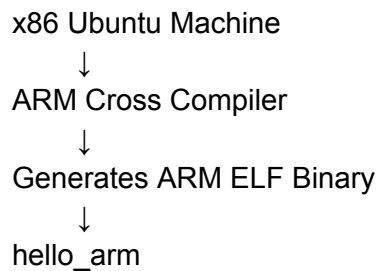
Compile ARM Binary

```
arm-linux-gnueabi-gcc hello.c -o hello_arm
```

PURPOSE OF THIS COMMAND

Part	Meaning
<code>arm-linux-gnueabihf-gcc</code>	ARM cross compiler
<code>hello.c</code>	Input source code
<code>-o hello_arm</code>	Output binary name

WHAT HAPPENS INTERNALLY



Verify Binary Architecture

file hello_arm

Purpose

Checks binary type and architecture.

Expected Output

hello_arm: ELF 32-bit LSB executable, ARM

Meaning

Word	Meaning
ELF	Executable binary format
32-bit	ARM32 architecture
ARM	Compiled for ARM CPU
executable	Runnable binary

STEP 5 — Execute ARM Binary Using QEMU

Install ARM Runtime Libraries

```
sudo apt install -y libc6-armhf-cross
```

Purpose

Installs ARM runtime libraries required by ARM applications.

Why Needed

Your ARM binary depends on ARM libc runtime files.

Without this:

No such file or directory

or

missing shared library

errors occur.

Run ARM Binary

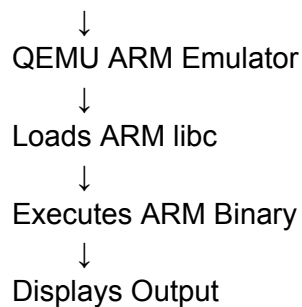
```
qemu-arm -L /usr/arm-linux-gnueabi/ ./hello_arm
```

PURPOSE OF EACH PART

Part	Meaning
<code>qemu-arm</code>	ARM userspace emulator
<code>-L</code>	Specifies ARM root filesystem path
<code>/usr/arm-linux-gnueabi</code> <code>hf/</code>	ARM runtime libraries location
<code>./hello_arm</code>	ARM executable

WHAT HAPPENS INTERNALLY

Ubuntu x86 System



EXPECTED OUTPUT

ARM test success

REAL INDUSTRY PURPOSE

Technology	Real Usage
Cross Compiler	Build firmware for ARM devices
QEMU	Test firmware without hardware
ARM libc	Runtime support for ARM apps
Device Tree	Hardware configuration
U-Boot	Embedded bootloader
Embedded Linux	Routers, BMC, IoT, Automotive

COMPLETE FLOW ARCHITECTURE

C Source Code



ARM Cross Compiler



ARM Binary (ELF)



QEMU Emulator



ARM Runtime Libraries



Execution on Ubuntu

FINAL LEARNING OUTCOME

After completing these steps, you understand:

Skill

Outcome

Cross Compilation	Build ARM binaries on Ubuntu
ELF Understanding	Verify executable architecture
QEMU Usage	Run ARM code without hardware
ARM Runtime	Understand shared libraries
Embedded Linux Workflow	Industry firmware development flow

Linux Kernel Build for ARM — Purpose of Every Command

This process teaches how real embedded Linux teams build ARM kernels for:

- Hewlett Packard Enterprise iLO/BMC firmware
 - Cisco networking devices
 - Samsung Electronics embedded products
 - Automotive ECUs
 - IoT gateways
 - ARM development boards
 - Embedded Linux systems
-

MAIN GOAL OF THIS FLOW

Download Linux kernel source



Configure kernel for ARM hardware



Cross compile kernel on Ubuntu x86



Generate ARM Linux kernel image

COMPLETE END-TO-END FLOW

Ubuntu x86 Machine



Download Linux Source Code



Extract Kernel Source



Configure ARM Board Settings



Cross Compile Linux Kernel



Generate ARM Kernel Image

STEP 1 — CREATE PROJECT DIRECTORY

```
mkdir -p ~/ilo-training/day2
```

PURPOSE

Creates training/project directory structure.

WHAT HAPPENS INTERNALLY

Part	Meaning
<code>mkdir</code>	Create directory
<code>-p</code>	Create parent folders automatically
<code>~/ilo-training/day2</code>	Target path

WHY THIS IS NEEDED

Kernel source code contains:

- Thousands of files
- Drivers
- Architecture code
- Build outputs

So separate workspace is required.

OUTCOME

`~/ilo-training/day2`

Project workspace created.

MOVE INTO DIRECTORY

`cd ~/ilo-training/day2`

PURPOSE

Changes terminal working directory.

WHY THIS IS NEEDED

All downloads and kernel build files will stay inside this workspace.

INTERNAL FLOW

Terminal shell



Changes current directory



Future commands execute here

STEP 2 — DOWNLOAD LINUX KERNEL SOURCE

```
wget https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.10.224.tar.xz
```

MAIN PURPOSE

Downloads Linux kernel source code from official Linux servers.

WHAT IS LINUX KERNEL?

Linux kernel is the core OS engine responsible for:

Responsibility	Purpose
Process management	Runs applications
Memory management	RAM allocation
Device drivers	Hardware communication
Scheduler	CPU task handling

Networking

TCP/IP stack

Filesystems

Storage access

WHAT HAPPENS INTERNALLY

Ubuntu Machine



Connects to kernel.org



Downloads compressed kernel archive



Stores tar.xz file locally

PURPOSE OF EACH PART

Part	Meaning
<code>wget</code>	Command-line downloader
<code>https://...</code>	Linux kernel source URL
<code>linux-5.10.224.tar.xz</code>	Compressed kernel source archive

WHY VERSION 5.10?

Linux 5.10 is:

- Stable
 - Long-term support (LTS)
 - Common in embedded systems
 - Widely used in production firmware
-

OUTPUT

linux-5.10.224.tar.xz

Downloaded compressed source archive.

STEP 3 — EXTRACT KERNEL SOURCE

`tar -xf linux-5.10.224.tar.xz`

PURPOSE

Extracts Linux kernel source files.

WHAT IS INSIDE THIS ARCHIVE?

Contains:

- Linux kernel source code
- Drivers
- Architecture files
- Networking stack
- Memory manager
- Build system
- Filesystem code

WHAT HAPPENS INTERNALLY

Compressed tar.xz file



↓
tar utility extracts files
↓
Creates linux-5.10.224 directory

PURPOSE OF EACH PART

Part	Meaning
<code>tar</code>	Archive extraction utility
<code>-x</code>	Extract archive
<code>-f</code>	Specify filename
<code>linux-5.10.224.tar</code> <code>.xz</code>	Input archive

OUTPUT

linux-5.10.224/

Kernel source directory created.

STEP 4 — MOVE INTO KERNEL SOURCE

`cd linux-5.10.224`

PURPOSE

Moves terminal into Linux kernel source tree.

WHY THIS IS IMPORTANT

Kernel build system exists inside this directory.

Important files include:

File	Purpose
<code>Makefile</code>	Kernel build system
<code>arch/</code>	CPU architecture code
<code>drivers/</code>	Device drivers
<code>fs/</code>	Filesystems
<code>net/</code>	Networking stack
<code>kernel/</code>	Core kernel logic

STEP 5 — CONFIGURE ARM KERNEL

`make ARCH=arm versatile_defconfig`

THIS IS ONE OF THE MOST IMPORTANT COMMANDS

MAIN PURPOSE

Creates default ARM kernel configuration.

VERY IMPORTANT CONCEPT

Linux kernel supports:

- Intel x86
- AMD64
- ARM
- ARM64
- RISC-V
- PowerPC
- MIPS

Kernel needs to know:

Which CPU architecture?

Which board?

Which drivers?

Which hardware?

WHAT THIS COMMAND DOES

Loads predefined ARM board configuration



Generates .config file



Kernel now knows:

- CPU type
 - ARM settings
 - Drivers
 - Memory layout
-

PURPOSE OF EACH PART

Part	Meaning
<code>make</code>	Linux build system
<code>ARCH=arm</code>	Target architecture = ARM
<code>versatile_defconfig</code> <code>ig</code>	Default ARM board configuration

WHAT IS versatile_defconfig?

It is:

Predefined ARM development board configuration

Used commonly with:

- QEMU ARM emulation
 - Embedded Linux learning
 - ARM kernel testing
-

WHAT HAPPENS INTERNALLY

Kernel build system



Reads ARM default config



Copies settings



Creates .config file

OUTPUT

.config

This file contains:

- Kernel features
 - Driver selections
 - CPU settings
 - Filesystem support
 - Networking configuration
-

VERY IMPORTANT UNDERSTANDING

Without `.config`:

Kernel does NOT know:

- What to build
 - Which drivers to include
 - Which CPU architecture
-

STEP 6 — BUILD ARM LINUX KERNEL

`make -j4 ARCH=arm CROSS_COMPILE=arm-linux-gnueabi-`

THIS IS THE REAL KERNEL COMPILATION STEP

MAIN PURPOSE

Cross compile Linux kernel for ARM CPU.

BIG PICTURE THINKING

Ubuntu x86 machine



Build ARM Linux kernel



Generate ARM kernel image

PURPOSE OF EACH PART

Part	Meaning
<code>make</code>	Starts kernel build
<code>-j4</code>	Use 4 parallel CPU threads
<code>ARCH=arm</code>	Build for ARM architecture
<code>CROSS_COMPILE=arm-linux-gnueabi- ihf-</code>	Use ARM cross compiler tools

VERY IMPORTANT CONCEPT

Kernel build internally calls:

arm-linux-gnueabi-
gcc
arm-linux-gnueabi-
ld
arm-linux-gnueabi-
objcopy
arm-linux-gnueabi-
as

WHAT HAPPENS INTERNALLY

Kernel Source Files



Preprocessor



ARM Compiler



Assembler



Object Files (.o)



Linker



ARM Linux Kernel Image

WHY -j4?

-j4

Means:

Use 4 CPU cores simultaneously

Benefits:

- Faster compilation
- Parallel build execution

```
Kernel: arch/arm/boot/zImage is ready
ubuntu@ubuntu-VMware-Virtual-Platform:~/ilo-training/day2/linux-5.10.224$ ls
arch      CREDITS    fs          ipc          lib          mm            modules.order  sam
block     crypto     include     Kbuild       LICENSES     modules.builtin  Module.symvers  scr
certs     Documentation  init        Kconfig      MAINTAINERS  modules.builtin.modinfo  net            sec
COPYING   drivers    io_uring    kernel        Makefile     modules-only.symvers  README         sou
ubuntu@ubuntu-VMware-Virtual-Platform:~/ilo-training/day2/linux-5.10.224$ cd arch/arm/boo
bash: cd: arch/arm/boo: No such file or directory
ubuntu@ubuntu-VMware-Virtual-Platform:~/ilo-training/day2/linux-5.10.224$ cd arch/arm/boot
ubuntu@ubuntu-VMware-Virtual-Platform:~/ilo-training/day2/linux-5.10.224/arch/arm/boot$ ls
bootp  compressed  deflate_xip_data.sh  dts  Image  install.sh  Makefile  zImage
ubuntu@ubuntu-VMware-Virtual-Platform:~/ilo-training/day2/linux-5.10.224/arch/arm/boot$
```

WHAT IS GENERATED AFTER BUILD?

Important output files:

File	Purpose
<code>zImage</code>	Compressed ARM kernel
<code>vmlinux</code>	Uncompressed kernel
<code>.o</code> files	Object files
<code>System.map</code>	Kernel symbols
<code>.dtb</code>	Device tree blobs

IMPORTANT OUTPUT LOCATION

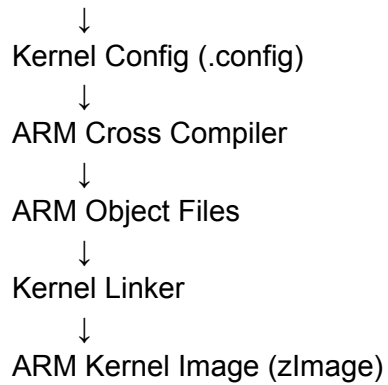
Usually:

`arch/arm/boot/zImage`

This is the ARM Linux kernel image.

COMPLETE INTERNAL BUILD FLOW

Linux Source Code



FINAL OUTCOME

You successfully created:

ARM Linux Kernel

using:

x86 Ubuntu Laptop

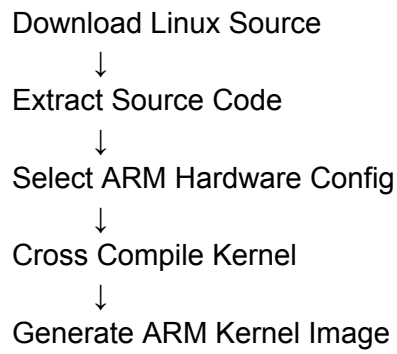
WITHOUT ARM hardware.

THIS IS EXACTLY HOW INDUSTRY WORKS

Industry Area	Real Usage
iLO Firmware	Build BMC Linux kernel
Routers	Build networking OS

Automotive	Build ECU kernel
IoT	Build ARM Linux images
Android	Build ARM phone kernel
Embedded Linux	Build BSP images

COMPLETE MENTAL MODEL



CORE ENGINEERING CONCEPTS YOU LEARNED

Concept	Meaning
Kernel Source	Linux OS core code
Cross Compilation	Build for another CPU
Defconfig	Default hardware config
ARM Architecture	Embedded CPU platform
Kernel Build System	make-based compilation
zImage	ARM compressed kernel

Toolchain

Compiler ecosystem

Parallel Build

Multi-core compilation

FINAL INDUSTRY UNDERSTANDING

This workflow is the foundation of:

- Embedded Linux engineering
- Firmware development
- Yocto BSP development
- ARM platform engineering
- iLO/BMC firmware
- Device driver development
- Linux kernel engineering
- Automotive Linux systems
- Networking firmware systems